

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 1

Roll no: 51 PRN No: 12411033 Name of Student: Manjiri Kulkarni

Subject Name : Deep Learning

Problem Statement :

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm :

1. Start.
2. Import the required libraries.
3. Load the Fashion MNIST dataset.
4. Normalize the image pixel values (0–255 to 0–1).
5. Display sample images from the dataset.
6. Create a Sequential neural network model.
7. Add Flatten, Dense (ReLU), and Dense (Softmax) layers.
8. Compile the model using the Adam optimizer and sparse categorical crossentropy loss.
9. Train the model using the training dataset.
10. Evaluate the model on the test dataset.
11. Display the test accuracy.
12. Stop.

Code :

```
import tensorflow as tf
```

```
import matplotlib.pyplot as plt
```

```
# Load Fashion MNIST dataset

from keras.datasets import fashion_mnist

# Load the Fashion MNIST dataset

(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

# Scale down pixel values from 0-255 to 0-1

train_images = train_images / 255.0

test_images = test_images / 255.0

# Print dataset details

print(train_images.shape)

print(test_images.shape)

print(train_labels)

# Fashion MNIST class names

class_names = ['T-shirt/Top', 'Trouser', 'Pullover', 'Dress',
               'Coat', 'Sandal', 'Shirt', 'Sneaker',
               'Bag', 'Ankle Boot']

# Display first image

plt.imshow(train_images[1], cmap='gray')

plt.title(class_names[train_labels[1]])
```

```
plt.show()

# Defining neural network model

my_model = tf.keras.models.Sequential()

my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))

my_model.add(tf.keras.layers.Dense(128, activation='relu'))

my_model.add(tf.keras.layers.Dense(10, activation='softmax'))


# Compile the model

my_model.compile(

    optimizer='adam',

    loss='sparse_categorical_crossentropy',

    metrics=['accuracy']

)

# Train the model

my_model.fit(train_images, train_labels, epochs=3)

# Evaluate the model

val_loss, val_acc = my_model.evaluate(test_images, test_labels)

print("Test Accuracy:", val_acc)
```

Output :

```
# Train the model
my_model.fit(train_images, train_labels, epochs=3)
```

```
... Epoch 1/3
1875/1875 ————— 12s 6ms/step - accuracy: 0.8246 - loss: 0.5029
Epoch 2/3
1875/1875 ————— 7s 4ms/step - accuracy: 0.8644 - loss: 0.3765
Epoch 3/3
1875/1875 ————— 8s 4ms/step - accuracy: 0.8757 - loss: 0.3388
<keras.src.callbacks.history.History at 0x7dfa5a8a9340>
```

```
# Evaluate the model
val_loss, val_acc = my_model.evaluate(test_images, test_labels)

print("Test Accuracy:", val_acc)
```

```
313/313 ————— 1s 2ms/step - accuracy: 0.8695 - loss: 0.3716
Test Accuracy: 0.8694999814033508
```